DAILY ACTIVITY TRACKER

Full Stack Web Application - Project Source Code Submission

Backend: Java | Spring Boot 3 | Spring Data JPA | MySQL / H2

Frontend: HTML5 | CSS3 | JavaScript ES6+ | Bootstrap 5

1. Database Schema

1. Database Schema

SQL DDL - categories and activities tables with seed data.

```
-- ================================================================================
-- 1. EXTENSIONS
-- ================================================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- ================================================================================
-- 2. CATEGORIES TABLE
-- ================================================================================
CREATE TABLE IF NOT EXISTS categories (
    category_id BIGSERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    color VARCHAR(7) DEFAULT '#0d6efd', -- Hex color for UI badges
    description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_categories_name ON categories(name);

-- Pre-seeded standard categories
INSERT INTO categories (name, color, description) VALUES
    ('Study', '#0d6efd', 'Academic studies, homework, revisions'),
    ('College', '#6f42c1', 'Lectures, labs, submissions, college events'),
    ('Exercise', '#198754', 'Gym, workouts, sports, running'),
    ('Personal', '#0dcaf0', 'Habits, chores, reading, errands'),
    ('Work', '#fd7e14', 'Part-time jobs, projects, internships'),
    ('Other', '#6c757d', 'Miscellaneous tasks')
ON CONFLICT (name) DO NOTHING;

-- ================================================================================
-- 3. ACTIVITIES TABLE (Core Entity)
-- ================================================================================
CREATE TABLE IF NOT EXISTS activities (
    activity_id BIGSERIAL PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    activity_date DATE NOT NULL,
    start_time TIME,
    end_time TIME,
    priority VARCHAR(10) NOT NULL DEFAULT 'MEDIUM'
        CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH')),
    status VARCHAR(15) NOT NULL DEFAULT 'PENDING'
        CHECK (status IN ('PENDING', 'COMPLETED')),
    category_id BIGINT REFERENCES categories(category_id) ON DELETE SET NULL,
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT chk_time_window CHECK (start_time IS NULL OR end_time IS NULL OR start_time <= end_time)
);

-- Performance Indexes
CREATE INDEX IF NOT EXISTS idx_activities_date ON activities(activity_date);
CREATE INDEX IF NOT EXISTS idx_activities_status ON activities(status);
CREATE INDEX IF NOT EXISTS idx_activities_category ON activities(category_id);
CREATE INDEX IF NOT EXISTS idx_activities_date_status ON activities(activity_date, status);

-- Trigger to automatically update updated_at on record changes
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
```

Application Screenshots

The following screenshots show the running Daily Activity Tracker application across all major views, along with the live database records.

Figure 1 - Manage Activities: Full list view with search bar, status/priority/category filters, edit and delete actions.

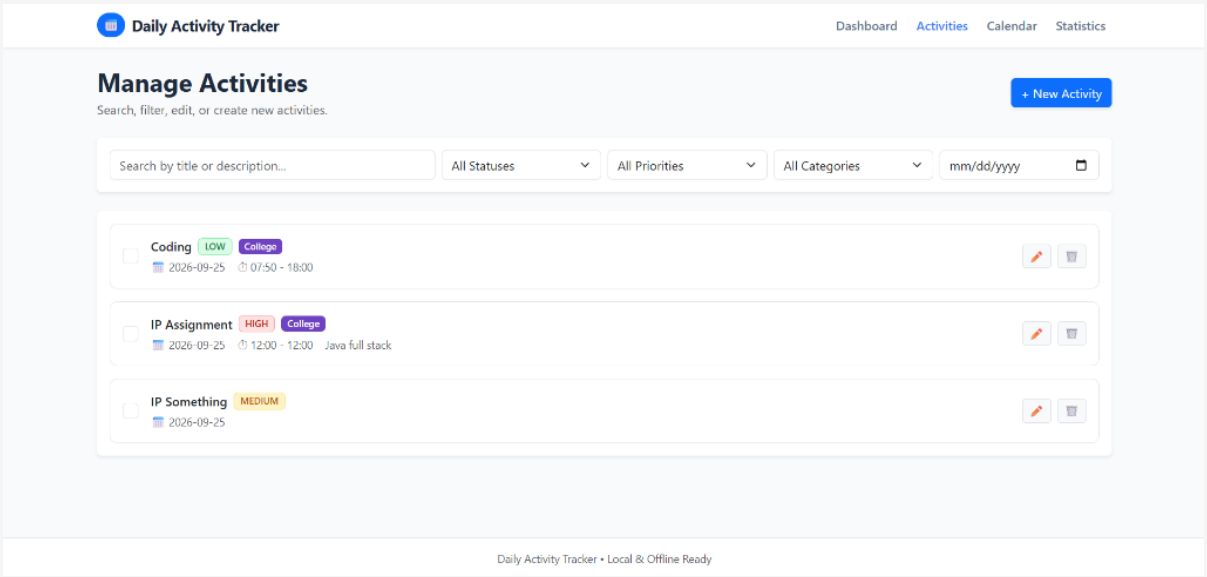


Figure 2 - Add Activity Modal: Create a task with title, description, date, start/end time, priority and category.

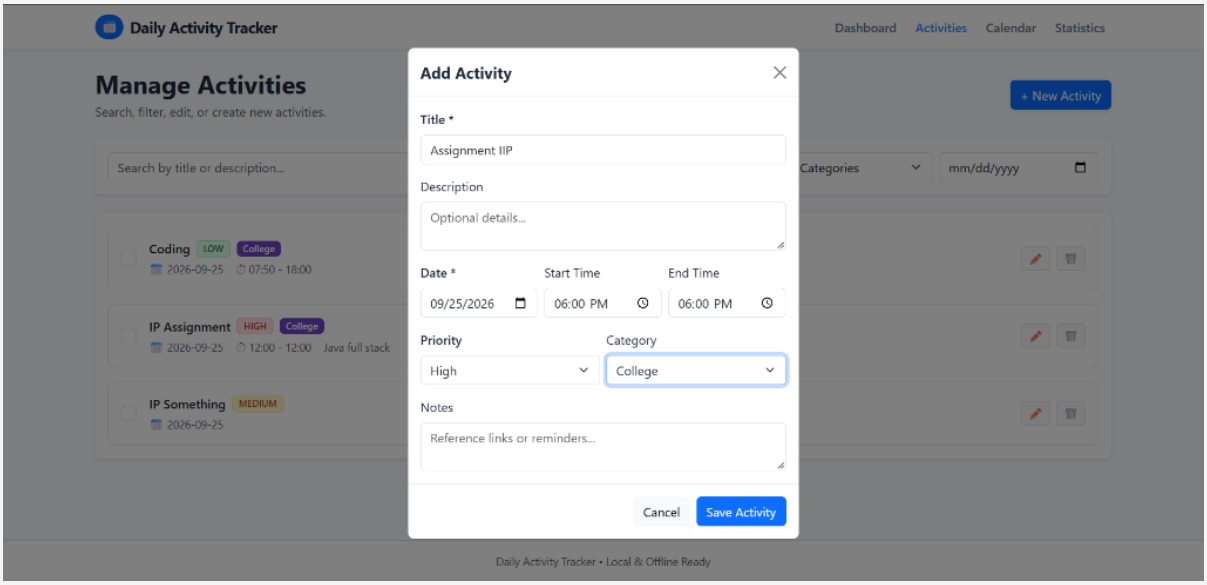


Figure 3 - Calendar View: Browse and inspect all activities scheduled by day with a monthly calendar grid.

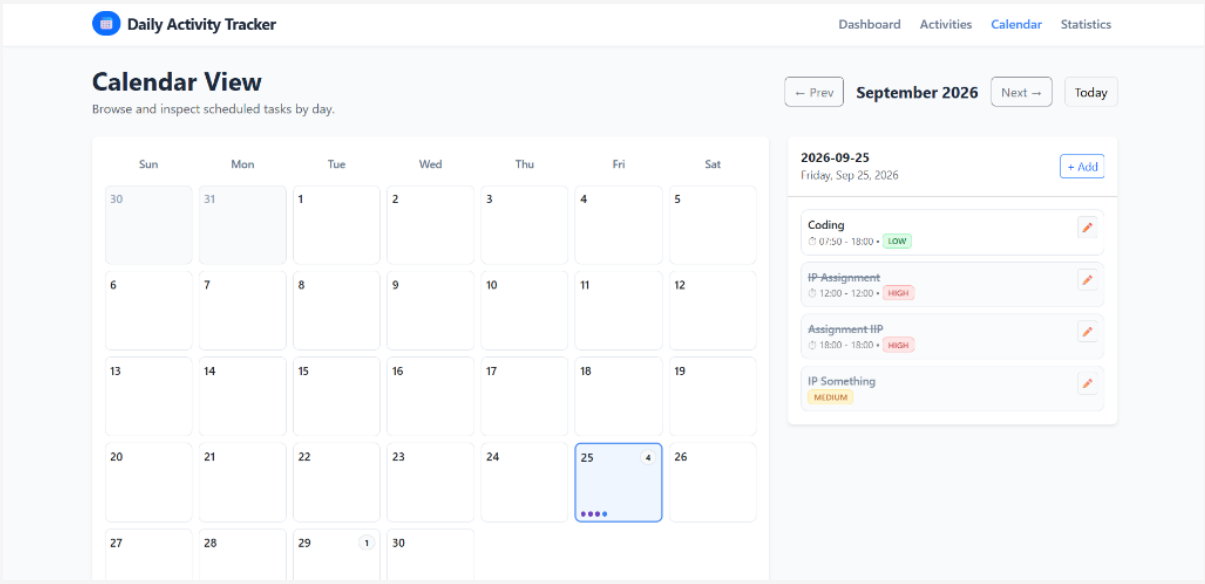


Figure 4 - Statistics & Analytics: Productivity metrics, category distribution, and priority breakdown charts.

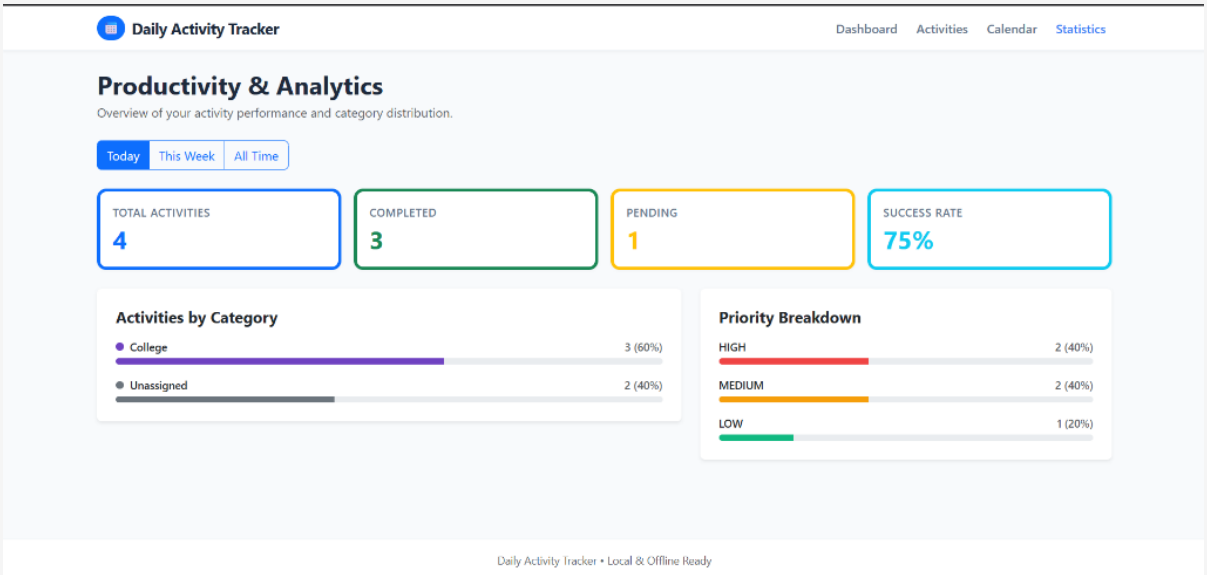
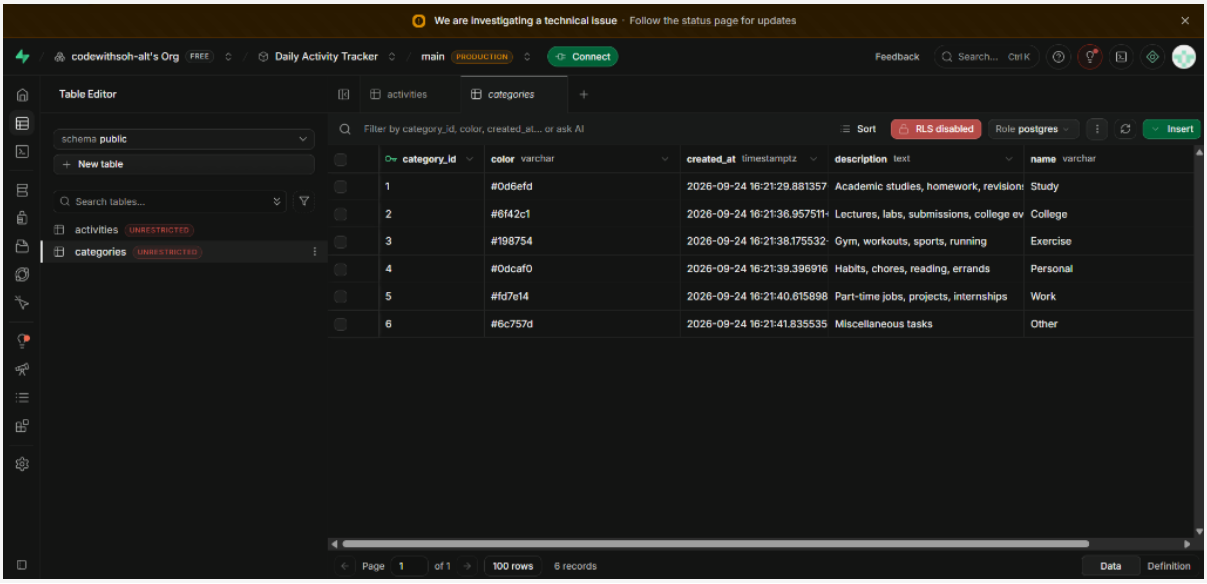


Figure 5 - Database (PostgreSQL / Supabase): Categories table showing category_id, color, description and name columns.



```
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trigger_activities_updated_at ON activities;
CREATE TRIGGER trigger_activities_updated_at
BEFORE UPDATE ON activities
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- ============================================================================
-- 4. RECURRING ACTIVITIES TABLE (Phase 3 Feature)
-- ============================================================================
CREATE TABLE IF NOT EXISTS recurring_activities (
    recurring_id BIGSERIAL PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    start_time TIME,
    end_time TIME,
    priority VARCHAR(10) NOT NULL DEFAULT 'MEDIUM'
        CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH')),
    category_id BIGINT REFERENCES categories(category_id) ON DELETE SET NULL,
    frequency VARCHAR(20) NOT NULL DEFAULT 'DAILY'
        CHECK (frequency IN ('DAILY', 'WEEKLY', 'MONTHLY')),
    days_of_week VARCHAR(30), -- e.g. 'MON,WED,FRI'
    start_date DATE NOT NULL,
    end_date DATE,
    active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT chk_recurring_dates CHECK (end_date IS NULL OR start_date <= end_date)
);

CREATE INDEX IF NOT EXISTS idx_recurring_active ON recurring_activities(active);
```

2. Backend Code (Java / Spring Boot)

2.1 Application Configuration

Datasource, JPA / Hibernate settings, and server port.

```
server.port=8080

# Supabase PostgreSQL Configuration (Project: evvkuhqkrvzvkmoxqqxn, Region: ap-southeast-2 Sydney)
# IPv4 Connection Pooler (Transaction mode on port 6543 with prepareThreshold=0 for PgBouncer)
spring.datasource.url=${DB_URL:jdbc:postgresql://aws-0-ap-southeast-
2.pooler.supabase.com:6543/postgres?sslmode=require&prepareThreshold=0}
spring.datasource.username=${DB_USERNAME:postgres.evvkuhqkrvzvkmoxqqxn}
spring.datasource.password=soham77890#
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / Hibernate Configuration
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Jackson Configuration
spring.jackson.date-format=yyyy-MM-dd
spring.jackson.time-zone=UTC
```

2.2 Main Application Class

Spring Boot entry point (@SpringBootApplication).

```
package com.activitytracker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ActivityTrackerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ActivityTrackerApplication.class, args);
    }
}
```

2.3a Model - Activity.java

JPA entity representing an activity record.

```
package com.activitytracker.model;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.OffsetDateTime;

@Entity
@Table(name = "activities")
public class Activity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "activity_id")
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "activity_date", nullable = false)
    private LocalDate activityDate;

    @Column(name = "start_time")
    private LocalTime startTime;

    @Column(name = "end_time")
    private LocalTime endTime;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private Priority priority = Priority.MEDIUM;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 15)
    private Status status = Status.PENDING;
```

```
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "category_id")
    private Category category;

    @Column(columnDefinition = "TEXT")
    private String notes;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt;

    public Activity() {
    }

    @PrePersist
    protected void onCreate() {
        OffsetDateTime now = OffsetDateTime.now();
        if (this.createdAt == null) {
            this.createdAt = now;
        }
        if (this.updatedAt == null) {
            this.updatedAt = now;
        }
        if (this.priority == null) {
            this.priority = Priority.MEDIUM;
        }
        if (this.status == null) {
            this.status = Status.PENDING;
        }
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = OffsetDateTime.now();
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }
```

```
public LocalDate getActivityDate() {
    return activityDate;
}

public void setActivityDate(LocalDate activityDate) {
    this.activityDate = activityDate;
}

public LocalTime getStartTime() {
    return startTime;
}

public void setStartTime(LocalTime startTime) {
    this.startTime = startTime;
}

public LocalTime getEndTime() {
    return endTime;
}

public void setEndTime(LocalTime endTime) {
    this.endTime = endTime;
}

public Priority getPriority() {
    return priority;
}

public void setPriority(Priority priority) {
    this.priority = priority;
}

public Status getStatus() {
    return status;
}

public void setStatus(Status status) {
    this.status = status;
}

public Category getCategory() {
    return category;
}

public void setCategory(Category category) {
    this.category = category;
}

public String getNotes() {
    return notes;
}

public void setNotes(String notes) {
    this.notes = notes;
}

public OffsetDateTime getCreatedAt() {
    return createdAt;
}
```

```
    public void setCreatedAt(OffsetDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public OffsetDateTime getUpdatedAt() {
        return updatedAt;
    }

    public void setUpdatedAt(OffsetDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }
}
```

2.3b Model - Category.java

JPA entity representing an activity category.

```
package com.activitytracker.model;

import jakarta.persistence.*;
import java.time.OffsetDateTime;

@Entity
@Table(name = "categories")
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "category_id")
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @Column(length = 7)
    private String color = "#0d6efd";

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt;

    public Category() {
    }

    public Category(Long id, String name, String color, String description) {
        this.id = id;
        this.name = name;
        this.color = color != null ? color : "#0d6efd";
        this.description = description;
    }

    @PrePersist
    protected void onCreate() {
        if (this.createdAt == null) {
            this.createdAt = OffsetDateTime.now();
        }
        if (this.color == null || this.color.isBlank()) {
            this.color = "#0d6efd";
        }
```

```
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public OffsetDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(OffsetDateTime createdAt) {
        this.createdAt = createdAt;
    }
}
```

2.4a Repository - ActivityRepository

Custom JPQL queries for filtering and statistics.

```
package com.activitytracker.repository;

import com.activitytracker.model.Activity;
import com.activitytracker.model.Priority;
import com.activitytracker.model.Status;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
```

```
@Repository
public interface ActivityRepository extends JpaRepository<Activity, Long> {

    List<Activity> findByActivityDateOrderByStartTimeAsc(LocalDate activityDate);

    List<Activity> findByActivityDateBetweenOrderByActivityDateAscStartTimeAsc(LocalDate startDate, LocalDate endDate);

    List<Activity> findByStatus(Status status);

    List<Activity> findByCategoryId(Long categoryId);

    // Overdue items: date before today and status PENDING
    @Query("SELECT a FROM Activity a WHERE a.activityDate < :today AND a.status = :status ORDER BY a.activityDate ASC,
a.startTime ASC")
    List<Activity> findOverdueActivities(@Param("today") LocalDate today, @Param("status") Status status);

    long countByActivityDate(LocalDate activityDate);

    long countByActivityDateAndStatus(LocalDate activityDate, Status status);

    long countByActivityDateBetween(LocalDate startDate, LocalDate endDate);

    long countByActivityDateBetweenAndStatus(LocalDate startDate, LocalDate endDate, Status status);

    long countByStatus(Status status);

    @Query("SELECT COUNT(a) FROM Activity a WHERE a.activityDate < :today AND a.status = :status")
    long countOverdueActivities(@Param("today") LocalDate today, @Param("status") Status status);

    @Query("SELECT a FROM Activity a WHERE " +
           "(:date IS NULL OR a.activityDate = :date) AND " +
           "(:status IS NULL OR a.status = :status) AND " +
           "(:priority IS NULL OR a.priority = :priority) AND " +
           "(:categoryId IS NULL OR a.category.id = :categoryId) AND " +
           "(CAST(:search AS string) IS NULL OR LOWER(a.title) LIKE LOWER(CONCAT('%', CAST(:search AS string), '%')) OR
LOWER(a.description) LIKE LOWER(CONCAT('%', CAST(:search AS string), '%'))) " +
           "ORDER BY a.activityDate DESC, a.startTime ASC")
    List<Activity> filterActivities(@Param("date") LocalDate date,
                                    @Param("status") Status status,
                                    @Param("priority") Priority priority,
                                    @Param("categoryId") Long categoryId,
                                    @Param("search") String search);
}
```

2.4b Repository - CategoryRepository

Spring Data JPA repository for categories.

```
package com.activitytracker.repository;

import com.activitytracker.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByNameIgnoreCase(String name);
    boolean existsByNameIgnoreCase(String name);
}
```

2.5a Service - ActivityService.java

Business logic: CRUD, status toggle, dashboard stats.

```
package com.activitytracker.service;

import com.activitytracker.dto.ActivityRequestDTO;
import com.activitytracker.dto.ActivityResponseDTO;
import com.activitytracker.dto.DashboardStatsDTO;
import com.activitytracker.exception.BadRequestException;
import com.activitytracker.exception.ResourceNotFoundException;
import com.activitytracker.model.Activity;
import com.activitytracker.model.Category;
import com.activitytracker.model.Priority;
import com.activitytracker.model.Status;
import com.activitytracker.repository.ActivityRepository;
import com.activitytracker.repository.CategoryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class ActivityService {

    private final ActivityRepository activityRepository;
    private final CategoryRepository categoryRepository;
    private final CategoryService categoryService;

    public ActivityService(ActivityRepository activityRepository,
                           CategoryRepository categoryRepository,
                           CategoryService categoryService) {
        this.activityRepository = activityRepository;
        this.categoryRepository = categoryRepository;
        this.categoryService = categoryService;
    }

    @Transactional(readOnly = true)
    public List<ActivityResponseDTO> getActivities(LocalDate date, Status status, Priority priority, Long categoryId,
String search) {
        return activityRepository.filterActivities(date, status, priority, categoryId, search).stream()
                .map(this::mapToResponseDTO)
                .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public ActivityResponseDTO getActivityById(Long id) {
        Activity activity = activityRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Activity not found with id: " + id));
        return mapToResponseDTO(activity);
    }

    @Transactional(readOnly = true)
    public List<ActivityResponseDTO> getActivitiesForDate(LocalDate date) {
        return activityRepository.findByActivityDateOrderByStartTimeAsc(date).stream()
                .map(this::mapToResponseDTO)
                .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
```

```
public List<ActivityResponseDTO> getOverdueActivities() {
    return activityRepository.findOverdueActivities(LocalDate.now(), Status.PENDING).stream()
            .map(this::mapToResponseDTO)
            .collect(Collectors.toList());
}

@Transactional
public ActivityResponseDTO createActivity(ActivityRequestDTO dto) {
    validateTimeWindow(dto);

    Activity activity = new Activity();
    applyDtoToActivity(dto, activity);

    Activity saved = activityRepository.save(activity);
    return mapToResponseDTO(saved);
}

@Transactional
public ActivityResponseDTO updateActivity(Long id, ActivityRequestDTO dto) {
    validateTimeWindow(dto);

    Activity activity = activityRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Activity not found with id: " + id));

    applyDtoToActivity(dto, activity);

    Activity updated = activityRepository.save(activity);
    return mapToResponseDTO(updated);
}

@Transactional
public ActivityResponseDTO updateStatus(Long id, Status status) {
    Activity activity = activityRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Activity not found with id: " + id));
    activity.setStatus(status);
    Activity updated = activityRepository.save(activity);
    return mapToResponseDTO(updated);
}

@Transactional
public void deleteActivity(Long id) {
    if (!activityRepository.existsById(id)) {
        throw new ResourceNotFoundException("Activity not found with id: " + id);
    }
    activityRepository.deleteById(id);
}

@Transactional(readOnly = true)
public DashboardStatsDTO getTodayStats() {
    LocalDate today = LocalDate.now();
    long total = activityRepository.countByActivityDate(today);
    long completed = activityRepository.countByActivityDateAndStatus(today, Status.COMPLETED);
    long pending = activityRepository.countByActivityDateAndStatus(today, Status.PENDING);
    long overdue = activityRepository.countOverdueActivities(today, Status.PENDING);

    double rate = (total > 0) ? Math.round(((double) completed / total) * 1000.0) / 10.0 : 0.0;
    return new DashboardStatsDTO(total, completed, pending, rate, overdue);
}

@Transactional(readOnly = true)
public DashboardStatsDTO getRangeStats(LocalDate start, LocalDate end) {
    long total = activityRepository.countByActivityDateBetween(start, end);
```

```
        long completed = activityRepository.countByActivityDateBetweenAndStatus(start, end, Status.COMPLETED);
        long pending = activityRepository.countByActivityDateBetweenAndStatus(start, end, Status.PENDING);
        long overdue = activityRepository.countOverdueActivities(LocalDate.now(), Status.PENDING);

        double rate = (total > 0) ? Math.round(((double) completed / total) * 1000.0) / 10.0 : 0.0;
        return new DashboardStatsDTO(total, completed, pending, rate, overdue);
    }

    @Transactional(readOnly = true)
    public DashboardStatsDTO getOverallStats() {
        long total = activityRepository.count();
        long completed = activityRepository.countByStatus(Status.COMPLETED);
        long pending = activityRepository.countByStatus(Status.PENDING);
        long overdue = activityRepository.countOverdueActivities(LocalDate.now(), Status.PENDING);

        double rate = (total > 0) ? Math.round(((double) completed / total) * 1000.0) / 10.0 : 0.0;
        return new DashboardStatsDTO(total, completed, pending, rate, overdue);
    }

    private void validateTimeWindow(ActivityRequestDTO dto) {
        if (dto.getStartTime() != null && dto.getEndTime() != null) {
            if (dto.getStartTime().isAfter(dto.getEndTime())) {
                throw new BadRequestException("Start time (" + dto.getStartTime() + ") must not be after end time (" +
dto.getEndTime() + ")");
            }
        }
    }

    private void applyDtoToActivity(ActivityRequestDTO dto, Activity activity) {
        activity.setTitle(dto.getTitle().trim());
        activity.setDescription(dto.getDescription());
        activity.setActivityDate(dto.getActivityDate());
        activity.setStartTime(dto.getStartTime());
        activity.setEndTime(dto.getEndTime());
        activity.setPriority(dto.getPriority() != null ? dto.getPriority() : Priority.MEDIUM);
        activity.setStatus(dto.getStatus() != null ? dto.getStatus() : Status.PENDING);
        activity.setNotes(dto.getNotes());

        if (dto.getCategoryId() != null) {
            Category category = categoryRepository.findById(dto.getCategoryId())
                    .orElseThrow(() -> new ResourceNotFoundException("Category not found with id: " +
dto.getCategoryId()));
            activity.setCategory(category);
        } else {
            activity.setCategory(null);
        }
    }

    public ActivityResponseDTO mapToResponseDTO(Activity activity) {
        if (activity == null) return null;
        ActivityResponseDTO dto = new ActivityResponseDTO();
        dto.setId(activity.getId());
        dto.setTitle(activity.getTitle());
        dto.setDescription(activity.getDescription());
        dto.setActivityDate(activity.getActivityDate());
        dto.setStartTime(activity.getStartTime());
        dto.setEndTime(activity.getEndTime());
        dto.setPriority(activity.getPriority());
        dto.setStatus(activity.getStatus());
        dto.setNotes(activity.getNotes());
        dto.setCategory(categoryService.mapToDTO(activity.getCategory()));
        dto.setCreatedAt(activity.getCreatedAt());
        dto.setUpdatedAt(activity.getUpdatedAt());
```

```
        // Overdue condition: activityDate is strictly before today and status is PENDING
        boolean isOverdue = activity.getStatus() == Status.PENDING &&
activity.getActivityDate().isBefore(LocalDate.now());
        dto.setOverdue(isOverdue);

        return dto;
    }
}
```

2.5b Service - CategoryService.java

Category CRUD and DTO mapping.

```
package com.activitytracker.service;

import com.activitytracker.dto.CategoryDTO;
import com.activitytracker.exception.BadRequestException;
import com.activitytracker.exception.ResourceNotFoundException;
import com.activitytracker.model.Category;
import com.activitytracker.repository.CategoryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class CategoryService {

    private final CategoryRepository categoryRepository;

    public CategoryService(CategoryRepository categoryRepository) {
        this.categoryRepository = categoryRepository;
    }

    @Transactional(readOnly = true)
    public List<CategoryDTO> getAllCategories() {
        return categoryRepository.findAll().stream()
                .map(this::mapToDTO)
                .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public CategoryDTO getCategoryById(Long id) {
        Category category = categoryRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Category not found with id: " + id));
        return mapToDTO(category);
    }

    @Transactional
    public CategoryDTO createCategory(CategoryDTO categoryDTO) {
        if (categoryRepository.existsByNameIgnoreCase(categoryDTO.getName())) {
            throw new BadRequestException("Category with name '" + categoryDTO.getName() + "' already exists");
        }
        Category category = new Category();
        category.setName(categoryDTO.getName().trim());
        category.setColor(categoryDTO.getColor() != null ? categoryDTO.getColor().trim() : "#0d6efd");
        category.setDescription(categoryDTO.getDescription());

        Category saved = categoryRepository.save(category);
        return mapToDTO(saved);
    }
```

```
    @Transactional
    public CategoryDTO updateCategory(Long id, CategoryDTO categoryDTO) {
        Category category = categoryRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Category not found with id: " + id));

        // Check unique name if altered
        categoryRepository.findByNameIgnoreCase(categoryDTO.getName().trim())
                .ifPresent(existing -> {
                    if (!existing.getId().equals(id)) {
                        throw new BadRequestException("Category with name '" + categoryDTO.getName() + "' already
exists");
                    }
                });

        category.setName(categoryDTO.getName().trim());
        if (categoryDTO.getColor() != null && !categoryDTO.getColor().isBlank()) {
            category.setColor(categoryDTO.getColor().trim());
        }
        category.setDescription(categoryDTO.getDescription());

        Category updated = categoryRepository.save(category);
        return mapToDTO(updated);
    }

    @Transactional
    public void deleteCategory(Long id) {
        if (!categoryRepository.existsById(id)) {
            throw new ResourceNotFoundException("Category not found with id: " + id);
        }
        categoryRepository.deleteById(id);
    }

    public CategoryDTO mapToDTO(Category category) {
        if (category == null) return null;
        return new CategoryDTO(
                category.getId(),
                category.getName(),
                category.getColor(),
                category.getDescription()
        );
    }
}
```

2.6a Controller - ActivityController

REST endpoints: GET, POST, PUT, PATCH, DELETE for activities.

```
package com.activitytracker.controller;

import com.activitytracker.dto.ActivityRequestDTO;
import com.activitytracker.dto.ActivityResponseDTO;
import com.activitytracker.model.Priority;
import com.activitytracker.model.Status;
import com.activitytracker.service.ActivityService;
import jakarta.validation.Valid;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;
```

```
@RestController
@RequestMapping("/api/activities")
public class ActivityController {

    private final ActivityService activityService;

    public ActivityController(ActivityService activityService) {
        this.activityService = activityService;
    }

    @GetMapping
    public ResponseEntity<List<ActivityResponseDTO>> getActivities(
            @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date,
            @RequestParam(required = false) Status status,
            @RequestParam(required = false) Priority priority,
            @RequestParam(required = false) Long categoryId,
            @RequestParam(required = false) String search) {
        return ResponseEntity.ok(activityService.getActivities(date, status, priority, categoryId, search));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ActivityResponseDTO> getActivityById(@PathVariable Long id) {
        return ResponseEntity.ok(activityService.getActivityById(id));
    }

    @GetMapping("/today")
    public ResponseEntity<List<ActivityResponseDTO>> getTodayActivities() {
        return ResponseEntity.ok(activityService.getActivitiesForDate(LocalDate.now()));
    }

    @GetMapping("/date/{date}")
    public ResponseEntity<List<ActivityResponseDTO>> getActivitiesByDate(
            @PathVariable @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date) {
        return ResponseEntity.ok(activityService.getActivitiesForDate(date));
    }

    @GetMapping("/overdue")
    public ResponseEntity<List<ActivityResponseDTO>> getOverdueActivities() {
        return ResponseEntity.ok(activityService.getOverdueActivities());
    }

    @PostMapping
    public ResponseEntity<ActivityResponseDTO> createActivity(@Valid @RequestBody ActivityRequestDTO dto) {
        ActivityResponseDTO created = activityService.createActivity(dto);
        return new ResponseEntity<>(created, HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ActivityResponseDTO> updateActivity(
            @PathVariable Long id,
            @Valid @RequestBody ActivityRequestDTO dto) {
        ActivityResponseDTO updated = activityService.updateActivity(id, dto);
        return ResponseEntity.ok(updated);
    }

    @PatchMapping("/{id}/complete")
    public ResponseEntity<ActivityResponseDTO> markAsCompleted(@PathVariable Long id) {
        ActivityResponseDTO updated = activityService.updateStatus(id, Status.COMPLETED);
        return ResponseEntity.ok(updated);
    }
```

```
    @PatchMapping("/{id}/pending")
    public ResponseEntity<ActivityResponseDTO> markAsPending(@PathVariable Long id) {
        ActivityResponseDTO updated = activityService.updateStatus(id, Status.PENDING);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteActivity(@PathVariable Long id) {
        activityService.deleteActivity(id);
        return ResponseEntity.noContent().build();
    }
}
```

2.6b Controller - CategoryController

REST endpoints for category management.

```
package com.activitytracker.controller;

import com.activitytracker.dto.CategoryDTO;
import com.activitytracker.service.CategoryService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/categories")
public class CategoryController {

    private final CategoryService categoryService;

    public CategoryController(CategoryService categoryService) {
        this.categoryService = categoryService;
    }

    @GetMapping
    public ResponseEntity<List<CategoryDTO>> getAllCategories() {
        return ResponseEntity.ok(categoryService.getAllCategories());
    }

    @GetMapping("/{id}")
    public ResponseEntity<CategoryDTO> getCategoryById(@PathVariable Long id) {
        return ResponseEntity.ok(categoryService.getCategoryById(id));
    }

    @PostMapping
    public ResponseEntity<CategoryDTO> createCategory(@Valid @RequestBody CategoryDTO categoryDTO) {
        CategoryDTO created = categoryService.createCategory(categoryDTO);
        return new ResponseEntity<>(created, HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<CategoryDTO> updateCategory(@PathVariable Long id, @Valid @RequestBody CategoryDTO
categoryDTO) {
        CategoryDTO updated = categoryService.updateCategory(id, categoryDTO);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
```

```
    public ResponseEntity<Void> deleteCategory(@PathVariable Long id) {
        categoryService.deleteCategory(id);
        return ResponseEntity.noContent().build();
    }
}
```

2.6c Controller - StatisticsController

REST endpoints for today / week / range statistics.

```
package com.activitytracker.controller;

import com.activitytracker.dto.DashboardStatsDTO;
import com.activitytracker.service.ActivityService;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.temporal.TemporalAdjusters;

@RestController
@RequestMapping("/api/statistics")
public class StatisticsController {

    private final ActivityService activityService;

    public StatisticsController(ActivityService activityService) {
        this.activityService = activityService;
    }

    @GetMapping("/today")
    public ResponseEntity<DashboardStatsDTO> getTodayStats() {
        return ResponseEntity.ok(activityService.getTodayStats());
    }

    @GetMapping("/week")
    public ResponseEntity<DashboardStatsDTO> getWeekStats() {
        LocalDate today = LocalDate.now();
        LocalDate startOfWeek = today.with(TemporalAdjusters.previousOrSame(DayOfWeek.MONDAY));
        LocalDate endOfWeek = today.with(TemporalAdjusters.nextOrSame(DayOfWeek.SUNDAY));
        return ResponseEntity.ok(activityService.getRangeStats(startOfWeek, endOfWeek));
    }

    @GetMapping("/overview")
    public ResponseEntity<DashboardStatsDTO> getOverviewStats() {
        return ResponseEntity.ok(activityService.getOverallStats());
    }

    @GetMapping("/range")
    public ResponseEntity<DashboardStatsDTO> getCustomRangeStats(
            @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate start,
            @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate end) {
        return ResponseEntity.ok(activityService.getRangeStats(start, end));
    }
}
```

2.6d Exception - GlobalExceptionHandler

Centralised @RestControllerAdvice error handler.

```
package com.activitytracker.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", OffsetDateTime.now());
        body.put("status", HttpStatus.NOT_FOUND.value());
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());
        return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<Map<String, Object>> handleBadRequest(BadRequestException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", OffsetDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("error", "Bad Request");
        body.put("message", ex.getMessage());
        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationErrors(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.put(error.getField(), error.getDefaultMessage());
        }

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", OffsetDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("error", "Validation Failed");
        body.put("validationErrors", fieldErrors);
        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, Object>> handleIllegalArgument(IllegalArgumentException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", OffsetDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("error", "Bad Request");
        body.put("message", ex.getMessage());
        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }
```

```
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneralException(Exception ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", OffsetDateTime.now());
        body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
        body.put("error", "Internal Server Error");
        body.put("message", ex.getMessage());
        return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

3. Frontend Code (HTML / CSS / JavaScript)

3.1 Frontend - index.html (Dashboard)

Today stats cards, progress bar, overdue banner, activity list.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard — Daily Activity Tracker</title>
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-custom sticky-top">
    <div class="container">
      <a class="navbar-brand d-flex align-items-center gap-2" href="index.html">
        <span class="badge bg-primary rounded-pill p-2">🗓</span>
        <span>Daily Activity Tracker</span>
      </a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto gap-2">
          <li class="nav-item"><a class="nav-link active" href="index.html">Dashboard</a></li>
          <li class="nav-item"><a class="nav-link" href="activities.html">Activities</a></li>
          <li class="nav-item"><a class="nav-link" href="calendar.html">Calendar</a></li>
          <li class="nav-item"><a class="nav-link" href="statistics.html">Statistics</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- Main Content -->
  <main class="container my-4 flex-grow-1">
    <!-- Header -->
    <div class="d-flex flex-wrap justify-content-between align-items-center mb-4 gap-3">
      <div>
        <h2 class="fw-bold mb-1">Today's Overview</h2>
        <p class="text-muted mb-0" id="current-date-display">Loading date...</p>
      </div>
      <div>
        <a href="activities.html?action=new" class="btn btn-primary d-inline-flex align-items-center gap-2 shadow-sm">
          <span>+ Add Activity</span>
        </a>
      </div>
    </div>
```

```
    <!-- Summary Metrics Cards -->
    <div class="row g-3 mb-4">
      <div class="col-sm-6 col-lg-3">
        <div class="stat-card p-3 border-start border-4 border-primary">
          <div class="stat-label">Total Today</div>
          <div class="stat-value text-primary mt-1" id="stat-total">0</div>
        </div>
      </div>
      <div class="col-sm-6 col-lg-3">
        <div class="stat-card p-3 border-start border-4 border-success">
          <div class="stat-label">Completed</div>
          <div class="stat-value text-success mt-1" id="stat-completed">0</div>
        </div>
      </div>
      <div class="col-sm-6 col-lg-3">
        <div class="stat-card p-3 border-start border-4 border-warning">
          <div class="stat-label">Pending</div>
          <div class="stat-value text-warning mt-1" id="stat-pending">0</div>
        </div>
      </div>
      <div class="col-sm-6 col-lg-3">
        <div class="stat-card p-3 border-start border-4 border-danger">
          <div class="stat-label">Overdue Tasks</div>
          <div class="stat-value text-danger mt-1" id="stat-overdue">0</div>
        </div>
      </div>
    </div>

    <!-- Progress Bar -->
    <div class="card border-0 shadow-sm mb-4">
      <div class="card-body p-3">
        <div class="d-flex justify-content-between align-items-center mb-2">
          <span class="fw-semibold">Today's Completion Rate</span>
          <span class="fw-bold text-primary" id="completion-rate-text">0%</span>
        </div>
        <div class="progress" style="height: 12px;">
          <div id="completion-progress-bar" class="progress-bar bg-primary progress-bar-striped progress-bar-animated"
role="progressbar" style="width: 0%"></div>
        </div>
      </div>
    </div>

    <!-- Overdue Alert Section -->
    <div id="overdue-banner" class="alert alert-danger d-none shadow-sm mb-4 align-items-center justify-content-
between" role="alert">
      <div>
        <strong id="overdue-count-alert">0 tasks</strong> are overdue from previous days.
      </div>
      <a href="activities.html?filter=overdue" class="btn btn-sm btn-outline-danger">Review Overdue</a>
    </div>

    <!-- Today's Activities List -->
    <div class="card border-0 shadow-sm">
      <div class="card-header bg-white py-3 d-flex justify-content-between align-items-center">
        <h5 class="fw-bold mb-0">Scheduled for Today</h5>
        <a href="activities.html" class="btn btn-sm btn-outline-secondary">View All</a>
      </div>
      <div class="card-body p-3">
        <div id="today-activities-list" class="vstack gap-3">
          <div class="text-center py-4 text-muted">Loading today's tasks...</div>
        </div>
      </div>
    </div>
  </main>
```

```
  <footer class="bg-white border-top py-3 text-center text-muted small">
    Daily Activity Tracker &bull; Local & Offline Ready
  </footer>

  <script src="js/bootstrap.bundle.min.js"></script>
  <script src="js/api.js"></script>
  <script src="js/dashboard.js"></script>
</body>
</html>
```

3.2 Frontend - activities.html

Activity list with search, filters, and create/edit modal.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Activities — Daily Activity Tracker</title>
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-custom sticky-top">
    <div class="container">
      <a class="navbar-brand d-flex align-items-center gap-2" href="index.html">
        <span class="badge bg-primary rounded-pill p-2">🗓</span>
        <span>Daily Activity Tracker</span>
      </a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto gap-2">
          <li class="nav-item"><a class="nav-link" href="index.html">Dashboard</a></li>
          <li class="nav-item"><a class="nav-link active" href="activities.html">Activities</a></li>
          <li class="nav-item"><a class="nav-link" href="calendar.html">Calendar</a></li>
          <li class="nav-item"><a class="nav-link" href="statistics.html">Statistics</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- Main Content -->
  <main class="container my-4 flex-grow-1">
    <!-- Header -->
    <div class="d-flex flex-wrap justify-content-between align-items-center mb-4 gap-3">
      <div>
        <h2 class="fw-bold mb-1">Manage Activities</h2>
        <p class="text-muted mb-0">Search, filter, edit, or create new activities.</p>
      </div>
      <div>
        <button class="btn btn-primary d-inline-flex align-items-center gap-2 shadow-sm" onclick="openCreateModal()">
          <span>+ New Activity</span>
        </button>
      </div>
    </div>

    <!-- Filter & Search Toolbar -->
    <div class="card border-0 shadow-sm mb-4">
      <div class="card-body p-3">
```

```
        <div class="row g-2">
          <div class="col-md-4">
            <input type="text" id="search-input" class="form-control" placeholder="Search by title or description...">
          </div>
          <div class="col-sm-6 col-md-2">
            <select id="filter-status" class="form-select">
              <option value="">All Statuses</option>
              <option value="PENDING">Pending</option>
              <option value="COMPLETED">Completed</option>
            </select>
          </div>
          <div class="col-sm-6 col-md-2">
            <select id="filter-priority" class="form-select">
              <option value="">All Priorities</option>
              <option value="HIGH">High</option>
              <option value="MEDIUM">Medium</option>
              <option value="LOW">Low</option>
            </select>
          </div>
          <div class="col-sm-6 col-md-2">
            <select id="filter-category" class="form-select">
              <option value="">All Categories</option>
            </select>
          </div>
          <div class="col-sm-6 col-md-2">
            <input type="date" id="filter-date" class="form-control">
          </div>
        </div>
      </div>
    </div>

    <!-- Activities List -->
    <div class="card border-0 shadow-sm">
      <div class="card-body p-3">
        <div id="activities-list" class="vstack gap-3">
          <div class="text-center py-5 text-muted">Loading activities...</div>
        </div>
      </div>
    </div>
  </main>

  <!-- Activity Add / Edit Modal -->
  <div class="modal fade" id="activityModal" tabindex="-1" aria-labelledby="activityModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content">
        <form id="activityForm" onsubmit="saveActivity(event)">
          <div class="modal-header">
            <h5 class="modal-title fw-bold" id="activityModalLabel">Add Activity</h5>
            <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
          </div>
          <div class="modal-body vstack gap-3">
            <input type="hidden" id="activity-id">

            <div>
              <label for="form-title" class="form-label fw-semibold">Title *</label>
              <input type="text" class="form-control" id="form-title" required maxlength="150" placeholder="e.g.,
Complete Chapter 3 Assignment">
            </div>

            <div>
              <label for="form-desc" class="form-label">Description</label>
              <textarea class="form-control" id="form-desc" rows="2" placeholder="Optional details..."></textarea>
            </div>
```

```
            <div class="row g-2">
              <div class="col-sm-4">
                <label for="form-date" class="form-label fw-semibold">Date *</label>
                <input type="date" class="form-control" id="form-date" required>
              </div>
              <div class="col-sm-4">
                <label for="form-start-time" class="form-label">Start Time</label>
                <input type="time" class="form-control" id="form-start-time">
              </div>
              <div class="col-sm-4">
                <label for="form-end-time" class="form-label">End Time</label>
                <input type="time" class="form-control" id="form-end-time">
              </div>
            </div>

            <div class="row g-2">
              <div class="col-sm-6">
                <label for="form-priority" class="form-label fw-semibold">Priority</label>
                <select class="form-select" id="form-priority">
                  <option value="LOW">Low</option>
                  <option value="MEDIUM" selected>Medium</option>
                  <option value="HIGH">High</option>
                </select>
              </div>
              <div class="col-sm-6">
                <label for="form-category" class="form-label">Category</label>
                <select class="form-select" id="form-category">
                  <option value="">None / Unassigned</option>
                </select>
              </div>
            </div>

            <div>
              <label for="form-notes" class="form-label">Notes</label>
              <textarea class="form-control" id="form-notes" rows="2" placeholder="Reference links or
reminders..."></textarea>
            </div>
          </div>
          <div class="modal-footer">
            <button type="button" class="btn btn-light" data-bs-dismiss="modal">Cancel</button>
            <button type="submit" class="btn btn-primary" id="saveActivityBtn">Save Activity</button>
          </div>
        </form>
      </div>
    </div>
  </div>

  <footer class="bg-white border-top py-3 text-center text-muted small">
    Daily Activity Tracker &bull; Local & Offline Ready
  </footer>

  <script src="js/bootstrap.bundle.min.js"></script>
  <script src="js/api.js"></script>
  <script src="js/activities.js"></script>
</body>
</html>
```

3.3 CSS - style.css

Custom styling: navbar, cards, badges, responsive layout.

```
:root {
  --primary-color: #3b82f6;
  --primary-hover: #2563eb;
```

```
  --bg-color: #f8fafc;
  --card-bg: #ffffff;
  --text-main: #1e293b;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
}

body {
  background-color: var(--bg-color);
  color: var(--text-main);
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

/* Navbar */
.navbar-custom {
  background-color: #ffffff;
  border-bottom: 1px solid var(--border-color);
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}

.navbar-brand {
  font-weight: 700;
  color: #1e293b !important;
  letter-spacing: -0.5px;
}

.nav-link {
  font-weight: 500;
  color: #64748b !important;
  transition: color 0.2s ease;
}

.nav-link:hover, .nav-link.active {
  color: var(--primary-color) !important;
}

/* Metric / Stat Cards */
.stat-card {
  border: 1px solid var(--border-color);
  border-radius: 12px;
  background: #ffffff;
  box-shadow: 0 1px 3px rgba(0,0,0,0.04);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.stat-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.08);
}

.stat-value {
  font-size: 2rem;
  font-weight: 700;
  line-height: 1.2;
}

.stat-label {
  font-size: 0.875rem;
  font-weight: 600;
  text-transform: uppercase;
```

```
  letter-spacing: 0.05em;
  color: var(--text-muted);
}

/* Activity Cards */
.activity-card {
  background: #ffffff;
  border: 1px solid var(--border-color);
  border-radius: 10px;
  transition: all 0.2s ease;
}

.activity-card:hover {
  border-color: #cbd5e1;
  box-shadow: 0 2px 4px rgba(0,0,0,0.04);
}

.activity-card.completed {
  background-color: #f8fafc;
  opacity: 0.85;
}

.activity-card.completed .activity-title {
  text-decoration: line-through;
  color: var(--text-muted);
}

.activity-card.overdue-border {
  border-left: 4px solid #ef4444 !important;
}

/* Badges */
.badge-priority-HIGH {
  background-color: #fee2e2;
  color: #b91c1c;
  font-weight: 600;
  border: 1px solid #fca5a5;
}

.badge-priority-MEDIUM {
  background-color: #fef3c7;
  color: #b45309;
  font-weight: 600;
  border: 1px solid #fde68a;
}

.badge-priority-LOW {
  background-color: #dcfce7;
  color: #15803d;
  font-weight: 600;
  border: 1px solid #86efac;
}

.badge-category {
  font-weight: 500;
  padding: 0.35em 0.65em;
  border-radius: 6px;
  color: #ffffff;
}

/* Calendar Styles */
.calendar-grid {
  display: grid;
```

```
  grid-template-columns: repeat(7, 1fr);
  gap: 8px;
}

.calendar-header-day {
  text-align: center;
  font-weight: 600;
  color: var(--text-muted);
  font-size: 0.875rem;
  padding: 8px 0;
}

.calendar-cell {
  aspect-ratio: 1.1;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 6px;
  background: #ffffff;
  cursor: pointer;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  transition: background-color 0.15s, border-color 0.15s;
}

.calendar-cell:hover {
  background-color: #f1f5f9;
}

.calendar-cell.active-day {
  border: 2px solid var(--primary-color);
  background-color: #eff6ff;
}

.calendar-cell.other-month {
  color: #94a3b8;
  background-color: #f8fafc;
}

.calendar-cell .day-num {
  font-size: 0.9rem;
  font-weight: 600;
}

.calendar-cell .task-dot-container {
  display: flex;
  gap: 3px;
  flex-wrap: wrap;
}

.task-dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background-color: var(--primary-color);
}

/* Alert Notification Toast/Banner */
#api-alert-container {
  position: fixed;
  top: 20px;
  right: 20px;
  z-index: 1060;
```

```
  min-width: 320px;
}
```

3.4 JS - api.js

Centralised Fetch API wrapper + toast notifications.

```
// Use relative '/api' when hosted on Render/production, or fallback to localhost during local file testing
const BASE_URL = window.location.origin.startsWith('http')
  ? `${window.location.origin}/api`
  : 'http://localhost:8080/api';

class ApiClient {
  static async request(endpoint, options = {}) {
    const url = `${BASE_URL}${endpoint}`;
    const defaultHeaders = {
      'Content-Type': 'application/json',
      'Accept': 'application/json'
    };

    const config = {
      ...options,
      headers: {
        ...defaultHeaders,
        ...(options.headers || {})
      }
    };

    try {
      const response = await fetch(url, config);

      if (response.status === 204) {
        return null;
      }

      const data = await response.json().catch(() => null);

      if (!response.ok) {
        const errorMsg = (data && (data.message || data.error)) || `HTTP ${response.status}: ${response.statusText}`;
        throw new Error(errorMsg);
      }

      return data;
    } catch (error) {
      console.error(`API Error on ${options.method || 'GET'} ${endpoint}:`, error);
      ApiClient.showToast(error.message || 'Network error occurred while contacting the server', 'danger');
      throw error;
    }
  }

  // --- Activities API ---
  static async getActivities(params = {}) {
    const searchParams = new URLSearchParams();
    if (params.date) searchParams.append('date', params.date);
    if (params.status) searchParams.append('status', params.status);
    if (params.priority) searchParams.append('priority', params.priority);
    if (params.categoryId) searchParams.append('categoryId', params.categoryId);
    if (params.search) searchParams.append('search', params.search);

    const query = searchParams.toString() ? `?${searchParams.toString()}` : '';
    return await ApiClient.request(`/activities${query}`);
  }
```

```
static async getTodayActivities() {
  return await ApiClient.request('/activities/today');
}

static async getActivitiesByDate(dateStr) {
  return await ApiClient.request(`/activities/date/${dateStr}`);
}

static async getOverdueActivities() {
  return await ApiClient.request('/activities/overdue');
}

static async getActivityById(id) {
  return await ApiClient.request(`/activities/${id}`);
}

static async createActivity(activityData) {
  return await ApiClient.request('/activities', {
    method: 'POST',
    body: JSON.stringify(activityData)
  });
}

static async updateActivity(id, activityData) {
  return await ApiClient.request(`/activities/${id}`, {
    method: 'PUT',
    body: JSON.stringify(activityData)
  });
}

static async markComplete(id) {
  return await ApiClient.request(`/activities/${id}/complete`, {
    method: 'PATCH'
  });
}

static async markPending(id) {
  return await ApiClient.request(`/activities/${id}/pending`, {
    method: 'PATCH'
  });
}

static async deleteActivity(id) {
  return await ApiClient.request(`/activities/${id}`, {
    method: 'DELETE'
  });
}

// --- Categories API ---
static async getCategories() {
  return await ApiClient.request('/categories');
}

static async createCategory(categoryData) {
  return await ApiClient.request('/categories', {
    method: 'POST',
    body: JSON.stringify(categoryData)
  });
}

// --- Statistics API ---
```

```
  static async getTodayStats() {
    return await ApiClient.request('/statistics/today');
  }

  static async getWeekStats() {
    return await ApiClient.request('/statistics/week');
  }

  static async getOverviewStats() {
    return await ApiClient.request('/statistics/overview');
  }

  static async getRangeStats(start, end) {
    return await ApiClient.request(`/statistics/range?start=${start}&end=${end}`);
  }

  // --- UI Toast Helper ---
  static showToast(message, type = 'info') {
    let container = document.getElementById('api-alert-container');
    if (!container) {
      container = document.createElement('div');
      container.id = 'api-alert-container';
      document.body.appendChild(container);
    }

    const alert = document.createElement('div');
    alert.className = `alert alert-${type} alert-dismissible fade show shadow-sm`;
    alert.role = 'alert';
    alert.innerHTML = `
      <div>${message}</div>
      <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    `;
    container.appendChild(alert);

    setTimeout(() => {
      alert.classList.remove('show');
      setTimeout(() => alert.remove(), 250);
    }, 4500);
  }
}
```

3.5 JS - dashboard.js

Loads stats, today list, overdue banner, status toggle.

```
document.addEventListener('DOMContentLoaded', async () => {
  displayTodayDate();
  await loadDashboard();
});

function displayTodayDate() {
  const dateElement = document.getElementById('current-date-display');
  if (dateElement) {
    const now = new Date();
    const options = { weekday: 'long', year: 'numeric', month: 'long', day: 'numeric' };
    dateElement.textContent = now.toLocaleDateString(undefined, options);
  }
}

async function loadDashboard() {
  try {
    const [stats, activities, overdueActivities] = await Promise.allSettled([
      ApiClient.getTodayStats(),
```

```
      ApiClient.getTodayActivities(),
      ApiClient.getOverdueActivities()
    ]);

    if (stats.status === 'fulfilled' && stats.value) {
      renderStats(stats.value);
    }

    if (activities.status === 'fulfilled' && Array.isArray(activities.value)) {
      renderTodayActivities(activities.value);
    }

    if (overdueActivities.status === 'fulfilled' && Array.isArray(overdueActivities.value)) {
      renderOverdueAlert(overdueActivities.value.length);
    }
  } catch (err) {
    console.error('Failed to load dashboard data:', err);
  }
}

function renderStats(stats) {
  document.getElementById('stat-total').textContent = stats.total ?? 0;
  document.getElementById('stat-completed').textContent = stats.completed ?? 0;
  document.getElementById('stat-pending').textContent = stats.pending ?? 0;
  document.getElementById('stat-overdue').textContent = stats.overdueCount ?? 0;

  const rate = Math.round(stats.completionRate || 0);
  document.getElementById('completion-rate-text').textContent = `${rate}%`;
  const progressBar = document.getElementById('completion-progress-bar');
  if (progressBar) {
    progressBar.style.width = `${rate}%`;
    progressBar.setAttribute('aria-valuenow', rate);
  }
}

function renderOverdueAlert(count) {
  const banner = document.getElementById('overdue-banner');
  const countText = document.getElementById('overdue-count-alert');
  if (!banner || !countText) return;

  if (count > 0) {
    countText.textContent = `${count} task${count > 1 ? 's' : ''}`;
    banner.classList.remove('d-none');
    banner.classList.add('d-flex');
  } else {
    banner.classList.add('d-none');
    banner.classList.remove('d-flex');
  }
}

function renderTodayActivities(activities) {
  const container = document.getElementById('today-activities-list');
  if (!container) return;

  if (activities.length === 0) {
    container.innerHTML = `
      <div class="text-center py-5">
        <p class="text-muted mb-2">No activities scheduled for today.</p>
        <a href="activities.html?action=new" class="btn btn-sm btn-outline-primary">+ Add an Activity</a>
      </div>
    `;
    return;
  }
```

```
  container.innerHTML = activities.map(activity => {
    const isCompleted = activity.status === 'COMPLETED';
    const timeFormatted = formatTimeWindow(activity.startTime, activity.endTime);
    const categoryBadge = activity.category ? `
      <span class="badge badge-category" style="background-color: ${activity.category.color || '#3b82f6'};">
        ${escapeHtml(activity.category.name)}
      </span>
    ` : '';

    return `
      <div class="activity-card p-3 d-flex align-items-center justify-content-between gap-3 ${isCompleted ? 'completed'
: ''} ${activity.overdue ? 'overdue-border' : ''}">
        <div class="d-flex align-items-center gap-3 flex-grow-1">
          <input type="checkbox" class="form-check-input mt-0 fs-5" style="cursor: pointer;"
            ${isCompleted ? 'checked' : ''}
            onchange="toggleTaskStatus(${activity.id}, this.checked)">
          <div>
            <div class="d-flex align-items-center gap-2 flex-wrap mb-1">
              <span class="fw-semibold activity-title fs-6">${escapeHtml(activity.title)}</span>
              <span class="badge badge-priority-${activity.priority}">${activity.priority}</span>
              ${categoryBadge}
            </div>
            <div class="small text-muted d-flex align-items-center gap-3">
              ${timeFormatted ? `<span>⏱ ${timeFormatted}</span>` : ''}
              ${activity.description ? `<span>${escapeHtml(activity.description)}</span>` : ''}
            </div>
          </div>
        </div>
        <div class="d-flex align-items-center gap-2">
          <a href="activities.html?edit=${activity.id}" class="btn btn-sm btn-light border" title="Edit">✎</a>
        </div>
      </div>
    `;
  }).join('');
}

async function toggleTaskStatus(id, isChecked) {
  try {
    if (isChecked) {
      await ApiClient.markComplete(id);
    } else {
      await ApiClient.markPending(id);
    }
    await loadDashboard();
  } catch (err) {
    console.error('Error toggling status:', err);
    await loadDashboard();
  }
}

function formatTimeWindow(start, end) {
  if (!start && !end) return '';
  if (start && !end) return `${start.slice(0, 5)}`;
  return `${start.slice(0, 5)} - ${end.slice(0, 5)}`;
}

function escapeHtml(str) {
  if (!str) return '';
  return str.replace(/[&<>"']/g, match => ({
    '&': '&amp;',
    '<': '&lt;',
    '>': '&gt;',
    '"': '&quot;',
```

```
    "'": '&#39;'
  }[match]));
}
```

3.6 JS - activities.js

Filter/search, render, create/edit modal, save, delete.

```
let activityModalInstance = null;
let currentCategories = [];

document.addEventListener('DOMContentLoaded', async () => {
  const modalEl = document.getElementById('activityModal');
  if (modalEl && window.bootstrap) {
    activityModalInstance = new bootstrap.Modal(modalEl);
  }

  setupEventListeners();
  await loadCategories();
  await handleUrlParams();
  await loadActivities();
});

function setupEventListeners() {
  const searchInput = document.getElementById('search-input');
  const filterStatus = document.getElementById('filter-status');
  const filterPriority = document.getElementById('filter-priority');
  const filterCategory = document.getElementById('filter-category');
  const filterDate = document.getElementById('filter-date');

  let debounceTimer;
  searchInput.addEventListener('input', () => {
    clearTimeout(debounceTimer);
    debounceTimer = setTimeout(loadActivities, 300);
  });

  filterStatus.addEventListener('change', loadActivities);
  filterPriority.addEventListener('change', loadActivities);
  filterCategory.addEventListener('change', loadActivities);
  filterDate.addEventListener('change', loadActivities);
}

async function handleUrlParams() {
  const urlParams = new URLSearchParams(window.location.search);
  if (urlParams.get('action') === 'new') {
    openCreateModal();
  } else if (urlParams.get('edit')) {
    const editId = urlParams.get('edit');
    await openEditModal(editId);
  } else if (urlParams.get('filter') === 'overdue') {
    // For overdue filter, default search to pending and clear date
    document.getElementById('filter-status').value = 'PENDING';
  }
}

async function loadCategories() {
  try {
    currentCategories = await ApiClient.getCategories() || [];
    populateCategoryDropdowns(currentCategories);
  } catch (err) {
    console.error('Failed to load categories:', err);
  }
}
```

```
function populateCategoryDropdowns(categories) {
  const filterSelect = document.getElementById('filter-category');
  const formSelect = document.getElementById('form-category');

  const filterOptions = categories.map(c => `<option value="${c.id}">${escapeHtml(c.name)}</option>`).join('');
  filterSelect.innerHTML = `<option value="">All Categories</option>${filterOptions}`;

  const formOptions = categories.map(c => `<option value="${c.id}">${escapeHtml(c.name)}</option>`).join('');
  formSelect.innerHTML = `<option value="">None / Unassigned</option>${formOptions}`;
}

async function loadActivities() {
  const container = document.getElementById('activities-list');
  container.innerHTML = '<div class="text-center py-4 text-muted">Loading activities...</div>';

  const params = {
    search: document.getElementById('search-input').value.trim(),
    status: document.getElementById('filter-status').value,
    priority: document.getElementById('filter-priority').value,
    categoryId: document.getElementById('filter-category').value,
    date: document.getElementById('filter-date').value
  };

  try {
    const activities = await ApiClient.getActivities(params) || [];
    renderActivities(activities);
  } catch (err) {
    container.innerHTML = `<div class="alert alert-danger">Error loading activities: ${escapeHtml(err.message)}</div>`;
  }
}

function renderActivities(activities) {
  const container = document.getElementById('activities-list');

  if (activities.length === 0) {
    container.innerHTML = `
      <div class="text-center py-5">
        <p class="text-muted mb-2">No matching activities found.</p>
        <button class="btn btn-sm btn-outline-primary" onclick="openCreateModal()">+ Create an Activity</button>
      </div>
    `;
    return;
  }

  container.innerHTML = activities.map(activity => {
    const isCompleted = activity.status === 'COMPLETED';
    const timeFormatted = formatTimeWindow(activity.startTime, activity.endTime);
    const categoryBadge = activity.category ? `
      <span class="badge badge-category" style="background-color: ${activity.category.color || '#3b82f6'};">
        ${escapeHtml(activity.category.name)}
      </span>
    ` : '';

    return `
      <div class="activity-card p-3 d-flex align-items-center justify-content-between gap-3 ${isCompleted ? 'completed'
: ''} ${activity.overdue ? 'overdue-border' : ''}">
        <div class="d-flex align-items-center gap-3 flex-grow-1">
          <input type="checkbox" class="form-check-input mt-0 fs-5" style="cursor: pointer;"
            ${isCompleted ? 'checked' : ''}
            onchange="toggleStatus(${activity.id}, this.checked)">
          <div>
            <div class="d-flex align-items-center gap-2 flex-wrap mb-1">
```

```
              <span class="fw-semibold activity-title fs-6">${escapeHtml(activity.title)}</span>
              <span class="badge badge-priority-${activity.priority}">${activity.priority}</span>
              ${categoryBadge}
              ${activity.overdue ? '<span class="badge bg-danger">Overdue</span>' : ''}
            </div>
            <div class="small text-muted d-flex align-items-center gap-3 flex-wrap">
              <span>📅 ${activity.activityDate}</span>
              ${timeFormatted ? `<span>⏱ ${timeFormatted}</span>` : ''}
              ${activity.description ? `<span>${escapeHtml(activity.description)}</span>` : ''}
              ${activity.notes ? `<span class="text-secondary fst-italic">📝 ${escapeHtml(activity.notes)}</span>` :
''}
            </div>
          </div>
        </div>
        <div class="d-flex align-items-center gap-2">
          <button class="btn btn-sm btn-light border" onclick="openEditModal(${activity.id})" title="Edit">✏</button>
          <button class="btn btn-sm btn-light border text-danger" onclick="deleteActivity(${activity.id})"
title="Delete">🗑</button>
        </div>
      </div>
    `;
  }).join('');
}

function openCreateModal() {
  document.getElementById('activityModalLabel').textContent = 'Add Activity';
  document.getElementById('activity-id').value = '';
  document.getElementById('form-title').value = '';
  document.getElementById('form-desc').value = '';
  document.getElementById('form-date').value = new Date().toISOString().split('T')[0];
  document.getElementById('form-start-time').value = '';
  document.getElementById('form-end-time').value = '';
  document.getElementById('form-priority').value = 'MEDIUM';
  document.getElementById('form-category').value = '';
  document.getElementById('form-notes').value = '';

  if (activityModalInstance) {
    activityModalInstance.show();
  }
}

async function openEditModal(id) {
  try {
    const activity = await ApiClient.getActivityById(id);
    if (!activity) return;

    document.getElementById('activityModalLabel').textContent = 'Edit Activity';
    document.getElementById('activity-id').value = activity.id;
    document.getElementById('form-title').value = activity.title || '';
    document.getElementById('form-desc').value = activity.description || '';
    document.getElementById('form-date').value = activity.activityDate || '';
    document.getElementById('form-start-time').value = activity.startTime ? activity.startTime.slice(0, 5) : '';
    document.getElementById('form-end-time').value = activity.endTime ? activity.endTime.slice(0, 5) : '';
    document.getElementById('form-priority').value = activity.priority || 'MEDIUM';
    document.getElementById('form-category').value = activity.category ? activity.category.id : '';
    document.getElementById('form-notes').value = activity.notes || '';

    if (activityModalInstance) {
      activityModalInstance.show();
    }
  } catch (err) {
    console.error('Failed to fetch activity for editing:', err);
  }
}
```

```
async function saveActivity(event) {
  event.preventDefault();
  const id = document.getElementById('activity-id').value;
  const startTimeVal = document.getElementById('form-start-time').value;
  const endTimeVal = document.getElementById('form-end-time').value;

  const payload = {
    title: document.getElementById('form-title').value.trim(),
    description: document.getElementById('form-desc').value.trim(),
    activityDate: document.getElementById('form-date').value,
    startTime: startTimeVal ? `${startTimeVal}:00` : null,
    endTime: endTimeVal ? `${endTimeVal}:00` : null,
    priority: document.getElementById('form-priority').value,
    categoryId: document.getElementById('form-category').value ? Number(document.getElementById('form-category').value)
: null,
    notes: document.getElementById('form-notes').value.trim()
  };

  try {
    if (id) {
      await ApiClient.updateActivity(id, payload);
      ApiClient.showToast('Activity updated successfully', 'success');
    } else {
      await ApiClient.createActivity(payload);
      ApiClient.showToast('Activity created successfully', 'success');
    }

    if (activityModalInstance) {
      activityModalInstance.hide();
    }
    await loadActivities();
  } catch (err) {
    console.error('Failed to save activity:', err);
  }
}

async function toggleStatus(id, isChecked) {
  try {
    if (isChecked) {
      await ApiClient.markComplete(id);
    } else {
      await ApiClient.markPending(id);
    }
    await loadActivities();
  } catch (err) {
    console.error('Error toggling status:', err);
    await loadActivities();
  }
}

async function deleteActivity(id) {
  if (!confirm('Are you sure you want to delete this activity?')) return;

  try {
    await ApiClient.deleteActivity(id);
    ApiClient.showToast('Activity deleted', 'info');
    await loadActivities();
  } catch (err) {
    console.error('Failed to delete activity:', err);
  }
}

function formatTimeWindow(start, end) {
```

```
  if (!start && !end) return '';
  if (start && !end) return `${start.slice(0, 5)}`;
  return `${start.slice(0, 5)} - ${end.slice(0, 5)}`;
}

function escapeHtml(str) {
  if (!str) return '';
  return str.replace(/[&<>"']/g, match => ({
    '&': '&amp;',
    '<': '&lt;',
    '>': '&gt;',
    '"': '&quot;',
    "'": '&#39;'
  }[match]));
}
```